# Running The Emulator

Put the following emulator files in a folder:

- Emulator.py
- EmulComputer.py
- EmulFPU.py

To start the emulator run:

- python3 <Path>Emulator.py [-h] <Path>Prog.mc <Path>TimmiOS.mc

where 'Prog.mc' contains the machine code for your program, i.e. the file that is is placed in Logisim's RAM, 'TimmiOS.mc' contains the operating system machine code, i.e. the file that is is placed in Logisim's ROM, and '-h' is an optional argument used to display the Emulator's usage.

For example:

```
Timmi: python3 ./Tools/Emulator/Emulator.py ./mc/Test.mc ./mc/TimmiOS.mc

Available Actions
-----------------
  R  - Run
  S  - Step
  I  - Interrupt
  RS - Reset
  SP - State Print
  MS - Memory Set
  MP - Memory Print
  BS - Breakpoint Set
  BD - Breakpoint Delete
  BP - Breakpoint Print
  GD - Graphics Dump
  H  - Help
  Q  - Quit

Action:
```

## The Run Command

Select ‘R’ to run the program to completion, or until a breakpoint is hit.

For example:

```
Action: R

R0:  0x7a8f, R1: 0x00fa, R2: 0x0000, R3: 0x0000, R4: 0x0000
IDX: 0x00fb, FP: 0x0000, LR: 0x0000, SP: 0xffef, PC: 0x800a

Flags - C: False, A: True, E: False, Z: True, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: False

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Program has completed - Perform Reset (RS) to re-run

Action:
```

## The Step Command

Select ’S’ to step through a program one instruction at a time.

For example:

```
Action: S

R0:  0x0000, R1: 0x0000, R2: 0x0000, R3: 0x0000, R4: 0x0000
IDX: 0x0000, FP: 0x0000, LR: 0x0000, SP: 0xffef, PC: 0x8001

Flags - C: False, A: False, E: True, Z: True, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: False

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Next Instr: CLR IDX (0xcdb4)

Action: S

R0:  0x0000, R1: 0x0000, R2: 0x0000, R3: 0x0000, R4: 0x0000
IDX: 0x0000, FP: 0x0000, LR: 0x0000, SP: 0xffef, PC: 0x8002

Flags - C: False, A: False, E: True, Z: True, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: False

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Next Instr: LDI R1,0x00fa (0x0004, 0x00fa)

Action:
```

## The Interrupt Command

Select 'I' to cause an interrupt, and then select the required interrupt:

1. ENTER_1 Button Interrupt
2. ENTER_2 Button Interrupt
3. Keyboard Interrupt

For example:

```
Next Instr: LDI R3,0xfff1 (0x000c, 0xfff1)

Action: I

Interrupt Number: 1

R0:  0x0000, R1: 0x0000, R2: 0x0000, R3: 0xfff1, R4: 0x0000
IDX: 0x0000, FP: 0x0000, LR: 0x0000, SP: 0xffef, PC: 0x0010

Flags - C: False, A: False, E: False, Z: False, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: True

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Next Instr: JMP 0xb000 (0x0800, 0xb000)

Action:
```

## The Reset Command

Select 'RS' to reset the program back to the beginning.

For example:

```
Action: RS

Computer Reset - Perform Run (R) or Step (S) to start

Action:
```

## The State Print Command

Select 'SP' to show the current state of the emulator.

For example:

```
Action: SP

R0:  0x0000, R1: 0x0000, R2: 0x0000, R3: 0xfff1, R4: 0x0000
IDX: 0x0000, FP: 0x0000, LR: 0x0000, SP: 0xffef, PC: 0x8003

Flags - C: False, A: False, E: False, Z: False, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: True

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Next Instr: LDI R4,0xfff2 (0x0010, 0xfff2)

Action:
```

## The Memory Commands

Select 'MP' to display a region of memory, and 'MS' to set a particular byte in memory.

The following example shows:

- The first 8 bytes of RAM (i.e. the start of the program)
- The top 4 bytes of the stack (it starts at address 0xffef & grows down)
- The contents of memory address 0x10a0 (start of 'os_read_str()' in ROM)

```
Action: MP

Starting Address: 0x8000
How many words: 8

Memory 0x8000: 0000 0012 0004 0007 101c 800a 0008 8100

Action: MP

Starting Address: 0xffec
How many words: 4

Memory 0xffec: 0000 8012 8012 8006

Action: MP

Starting Address: 0x10a0
How many words: 1

Memory 0x10a0: 8600

Action:
```

The following example show the contents of memory address 0x9000 being set:

```
Action: MP

Starting Address: 0x9000
How many words: 1

Memory 0x9000: 0000

Action: MS

Address: 0x9000
Val: 0x1234

Action: MP

Starting Address: 0x9000
How many words: 1

Memory 0x9000: 1234

Action:
```

## The Breakpoint Commands

Select ‘BS’ to set a breakpoint, ‘BD’ to delete one, and ‘BP’ to display the current breakpoints.

The following example shows the usage of these commands:

```
Action: BS

Add a breakpoint at PC = 0x8004

Action: BS

Add a breakpoint at PC = 0x800c

Action: BP

Current breakpoints: 0x8004 0x800c

Action: BD

Remove breakpoint at PC = 0x800c

Action: BP

Current breakpoints: 0x8004

Action:
```

When a breakpoint is hit the instruction that is about to be performed is shown, together with the current emulator state. In order to see the results of performing the command advance the program one step.

This example shows a program running until it hits a breakpoint:

```
Action: R

Hit breakpoint at PC = 0x8004

R0:  0x000a, R1: 0x00fa, R2: 0x0000, R3: 0x0000, R4: 0x0000
IDX: 0x0005, FP: 0x0000, LR: 0x0000, SP: 0xffef, PC: 0x8004

Flags - C: False, A: False, E: False, Z: True, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: False

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Next Instr: ADD R0,IDX,R0 (0x88a0)

Action:
```

## The Graphics Dump Command

Select 'GD' to save the current state of the Graphics Memory.

The file is saved as 'Graphics.dat' in the current working folder. The file can then be used as a 'background' within Logisim in order to view it.

For example:

```
Action: GD

Graphics dumped to 'Graphics.dat'

Action:
```

## The Help Command

Select 'H' to show the available actions.

For example:

```
Action: H

Available Actions
-----------------
  R  - Run
  S  - Step
  I  - Interrupt
  RS - Reset
  SP - State Print
  MS - Memory Set
  MP - Memory Print
  BS - Breakpoint Set
  BD - Breakpoint Delete
  BP - Breakpoint Print
  GD - Graphics Dump
  H  - Help
  Q  - Quit

Action:
```

## The Quit Command

Select 'Q' to quit the emulator.

For example:

```
Action: Q

Emulation Finished

Timmi:
```

## “Crashes”

The emulator can detect the following error conditions:

- Trying to load a program that is too big for memory
- Stack Overflow: Trying to PUSH to ROM
- Stack Underflow: Trying to POP from the I/O area of memory

These are shown in the following examples:

```
Timmi: python3 ./Tools/Emulator/Emulator.py ./mc/Test.mc ./mc/TimmiOS.mc

*** CRASH ***
Program too big for RAM

Timmi:
```

```
Action: s

R0:  0x0000, R1: 0x0000, R2: 0x0000, R3: 0x0000, R4: 0x0000
IDX: 0x0000, FP: 0x0000, LR: 0x0000, SP: 0x8000, PC: 0x8000

Flags - C: False, A: False, E: False, Z: False, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: False

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Next Instr: PUSH R1 (0x8100)

Action: s

*** CRASH ***
Stack Overflow - Stack Pointer now points to ROM

Timmi:
```

```
Action: s

R0:  0x0000, R1: 0x0000, R2: 0x0000, R3: 0x0000, R4: 0x0000
IDX: 0x0000, FP: 0x0000, LR: 0x0000, SP: 0xffef, PC: 0x8000

Flags - C: False, A: False, E: False, Z: False, N: False, V: False
FPU - UNF: False, OVF: False, INF: False, NaN: False
Interrupts Enabled: False

IN_1: 0x0000, WAIT_1: 0x0000, ENTER_1: 0x0000, OUT_1: 0x0000
IN_2: 0x0000, WAIT_2: 0x0000, ENTER_2: 0x0000, OUT_2: 0x0000

Next Instr: POP R1 (0x7804)

Action: s

*** CRASH ***
Stack Underflow - Stack Pointer now points to I/O memory

Timmi:
```

## Emulator & Hardware Differences

The ‘A’ and ‘E’ flags may differ between the Emulator and the ‘SAL-16’ Logisim ‘Hardware’. This can happen when single argument commands are issued, e.g. ‘INC R0’ or ‘NEG R2’.

The reason for this is because the ‘Hardware’ compares the RA and TMP registers when setting these flags, whereas the Emulator compares the RA and RB registers, as the TMP register is not simulated within the Emulator.

However, this should not be an issue as it isn’t logical to perform any ‘jumps’ based on these flags after using commands such as ‘INC R0’ or ‘NEG R2’.

For floating point operations there may be small differences in the results given by the Emulator and the Logisim ‘Hardware’ of the ‘SAL-16’ due to the differing ‘rounding errors’ accumulated in the Python code of the Emulator as against the ‘SAL-16’ Logisim ‘Hardware’.